

十大機器學習演算法

全方位深度研讀報告

A Beginner-to-Advanced Comprehensive Guide

從基礎數值預測到高階神經網絡架構解析

編撰：NCHU AI Training 教學模組

目標受眾：資料科學家、AI工程師、專案經理

日期：2026年 6月

前言：機器學習的核心範式 (Core Paradigms of Machine Learning)

機器學習 (Machine Learning, ML) 是人工智慧的核心驅動力，其根本目的在於「讓電腦從數據中學習規律，而不需要進行明確的程式規則撰寫」。在深入探討十大演算法之前，我們必須先理解機器學習的三大核心範式：

- **監督式學習 (Supervised Learning)**：這是目前工業界應用最廣泛的類型。資料集包含了「特徵 (Features)」與「標籤 (Labels)」。演算法的目標是學習特徵與標籤之間的對映關係，以便在未來遇到新的特徵時能預測出正確的標籤。主要任務分為「迴歸 (Regression，預測連續數值)」與「分類 (Classification，預測離散類別)」。
- **非監督式學習 (Unsupervised Learning)**：給予演算法的資料完全沒有標籤，演算法必須自行在資料的汪洋中尋找隱藏的結構、模式或群體。常見任務包括「分群 (Clustering)」與「降維 (Dimensionality Reduction)」。
- **強化學習 (Reinforcement Learning)**：透過讓智能體 (Agent) 在環境中採取行動，並根據環境給予的獎勵 (Reward) 或懲罰 (Penalty) 來優化其行為策略。

本深度研讀報告將依循漸進式學習路徑 (3-Step Progression)，為您詳細剖析當今產業界最常使用的十種核心演算法，並提供深度的理論拆解與實務案例。

第一階段：數值預測與基礎分類 (Predictive & Logic-Based Models)

1. 線性迴歸 (Linear Regression)

線性迴歸是統計學與機器學習中最古老、也是最基礎的演算法之一。它屬於監督式學習中的「迴歸」任務。其核心思想非常直觀：假設自變數（特徵 X ）與應變數（目標 Y ）之間存在著線性關係，並試圖在資料空間中擬合出一條「最佳直線」，以最小化預測值與實際值之間的誤差。

數學原理與核心機制

簡單線性迴歸的方程式可以表示為：

$$Y = \beta_0 + \beta_1 X + \varepsilon$$

其中， Y 是我們要預測的目標變數； X 是輸入的特徵； β_0 是截距 (Intercept)； β_1 是斜率 (係數)，代表 X 每變動一單位對 Y 造成的影響； ε 是無法被模型解釋的殘差 (誤差項)。

為了找到這條「最佳直線」，線性迴歸通常使用**最小平方法 (Ordinary Least Squares, OLS)**，其目標是最小化所有資料點到該直線的垂直距離平方和 (Sum of Squared Residuals)。

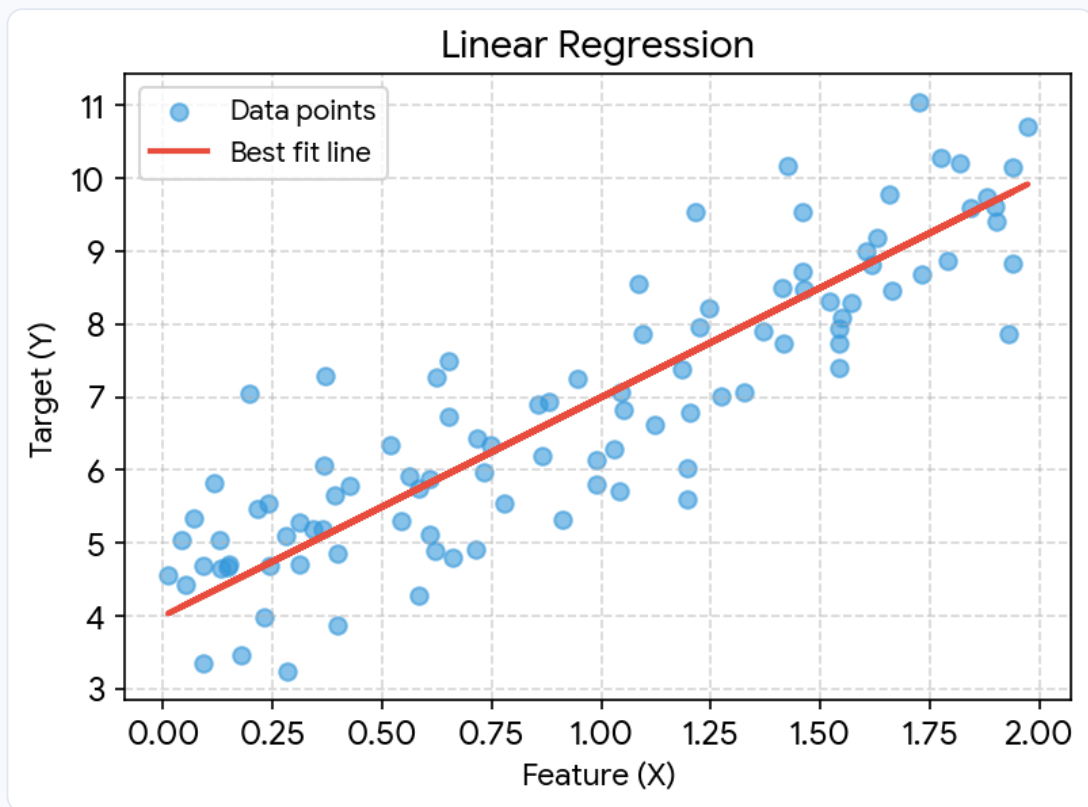


圖 1：線性迴歸擬合示意圖。紅線代表最小化誤差後的最佳擬合直線。

實務應用案例：房地產價格預測系統

在房地產估價中，房屋的價格（Y）通常與其坪數大小（X）呈現高度的正相關。一家房仲業者可以收集過去十年的歷史交易紀錄，使用線性迴歸建立模型。當有新的物件要上市時，只要輸入該物件的坪數，模型就能在擬合出的直線上找到對應的點，從而給出一個客觀的「建議售價」。進階的「多元線性迴歸」甚至能同時考量屋齡、捷運距離、樓層等多個變數，提供更精準的估價。

優點 (Pros)	缺點 (Cons)
✓ 模型極具解釋性，係數的意義一目了然。 ✓ 計算速度極快，訓練時間短。 ✓ 是了解更複雜模型的優良基石。	✗ 強烈依賴「線性」假設，若現實資料為非線性則表現極差。 ✗ 對極端值 (Outliers) 非常敏感，容易導致擬合線嚴重偏移。 ✗ 容易發生多重共線性問題 (特徵之間高度相關)。

2. 邏輯迴歸 (Logistic Regression)

儘管名字中帶有「迴歸」二字，邏輯迴歸實際上是一種強大的**分類 (Classification)**演算法，最常用於解決「二元分類 (Binary Classification)」問題（如：是/否、成功/失敗、正常/異常）。

數學原理與核心機制

線性迴歸的輸出範圍從負無限大到正無限大，這不適合用來表示機率。因此，邏輯迴歸引入了 **Sigmoid 函數**（或稱 Logistic 函數），將線性方程式的輸出強制壓縮 (Mapping) 到 0 到 1 之間的區間。這個值可以被解釋為某事件發生的「機率」。

$$p = 1 / (1 + e^{-(\beta_0 + \beta_1 X)})$$

當我們計算出機率 **p** 後，會設定一個決策閾值 (Threshold，通常為 0.5)。若 $p \geq 0.5$ ，將資料分類為類別 1；若 $p < 0.5$ ，則分類為類別 0。這使得它成為一個邊界清晰的二元分類器。

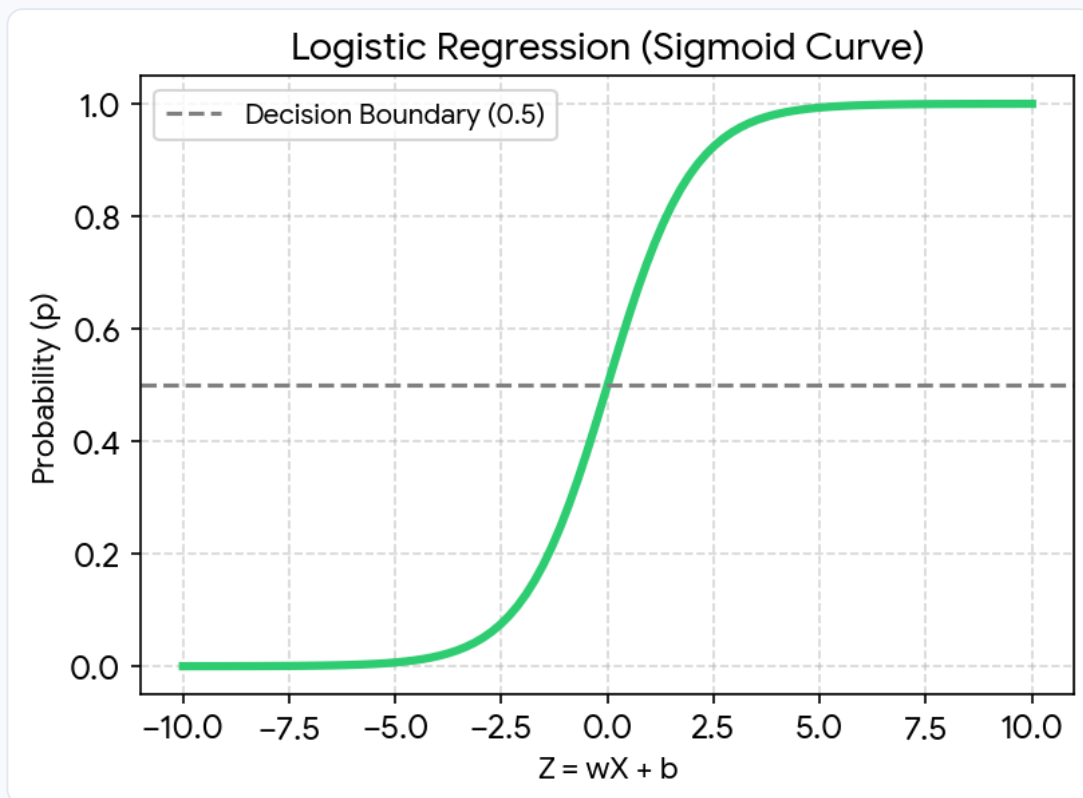


圖 2：Sigmoid 曲線。將任何實數輸入轉換為 0 到 1 之間的機率值。

實務應用案例：銀行信用卡核貸與違約預測

銀行需要評估客戶申請信用卡時，未來是否會發生違約。銀行會將客戶的年收入、過去信用評分、負債比等特徵 (X) 輸入邏輯迴歸模型中。模型最終會輸出一個介於 0 到 1 的數值，例如 0.85。這代表該客戶違約的「機率」高達 85%。因為 0.85 大於銀行設定的 0.5 閾值，系統便會自動將其分類為「高風險 (1)」，進而拒絕核卡或要求提供更多擔保品。

3. 決策樹 (Decision Tree)

決策樹是一種直觀、接近人類思考邏輯的監督式學習演算法，它同時具備處理分類與迴歸任務的能力。它透過一系列的「是/否」問題，將龐大且複雜的資料集切割成越來越小的子集，最終形成一個樹狀的決策模型。

數學原理與核心機制

一棵決策樹由「根節點 (Root Node)」、「內部決策節點 (Internal Nodes)」與「葉節點 (Leaf Nodes)」組成。演算法在生長過程中，必須決定在哪個特徵上進行切割。為了找到最佳切分點，決策樹依賴特定的數學指標來衡量純度：

- **資訊熵 (Entropy) 與 資訊增益 (Information Gain)：**熵代表系統的混亂程度。決策樹希望每次切割後，子節點的資料能越「純」越好（例如一邊全是患病、一邊全是健康）。切割前後熵的下降量稱為「資訊增益」，演算法會選擇資訊增益最大的特徵進行切割。
- **基尼不純度 (Gini Impurity)：**計算從資料集中隨機抽取兩個樣本，其類別標籤不一致的機率。CART 演算法常使用 Gini 來作為切分標準，計算速度較快。

實務應用案例：醫療分流輔助系統

在急診室中，決策樹可以作為醫師的初步分流輔助。根節點可能是判斷「病患是否有胸痛？」。若「是」，進入左子樹，下一個問題是「心電圖是否異常？」；若「否」，進入右子樹，判斷「是否有發燒？」。透過這樣層層遞進的 if-else 邏輯，最終葉節點會給出該病患的風險等級（如：立刻送入加護病房 vs. 在候診區等待）。其優勢在於，醫療人員可以完全看懂模型的推論過程，具備極高的透明度與可解釋性 (White-box model)。

4. 隨機森林 (Random Forest)

決策樹雖然解釋性強，但它有一個致命缺點：非常容易「過度擬合 (Overfitting)」。當樹長得太深時，它會把訓練資料中的雜訊也硬背下來，導致在面對新資料時預測極不準確。為了解決這個問題，**集成學習 (Ensemble Learning)** 的王者——隨機森林應運而生。

數學原理與核心機制

隨機森林的精髓在於「群體智慧 (Wisdom of the Crowd)」。它結合了兩個強大的隨機性機制：

1. **Bootstrap Aggregating (Bagging)：**從原始訓練資料集中，透過「隨機且放回」的方式，抽出多個不同的子資料集，用來訓練不同的決策樹。

2. **特徵隨機選擇 (Random Feature Selection)**：在每棵樹分裂節點時，不考慮所有的特徵，而是隨機挑選一部分特徵來尋找最佳切分點。

當這數百或數千棵彼此獨立且略有差異的決策樹建立好之後，面對一筆新資料，隨機森林會讓所有的樹進行預測。如果是分類問題，就採「多數決 (Majority Voting)」；如果是迴歸問題，就取所有樹預測值的「平均 (Average)」。這大幅抵銷了單一樹的偏差與變異。

第二階段：空間幾何與機率模型 (Spatial & Probabilistic Models)

5. 支援向量機 (Support Vector Machine, SVM)

SVM 是在深度學習普及前，最為輝煌且強大的傳統機器學習演算法之一。它以嚴謹的幾何數學為基礎，擅長在複雜、高維度的空間中畫出完美的分類邊界。

數學原理與核心機制

SVM 的核心目標非常純粹：尋找一個**最佳超平面 (Optimal Hyperplane)**，將不同類別的資料分開。但它不僅僅是分開，它還要追求「最寬的街道」。

這個街道的寬度被稱為 **Margin (邊界間距)**。SVM 會尋找距離超平面最近的那些關鍵資料點，這些點被稱為 **支援向量 (Support Vectors)**。超平面的位置完全由這些支援向量決定，演算法的數學優化目標，就是最大化這個 Margin，這賦予了模型極強的泛化能力。

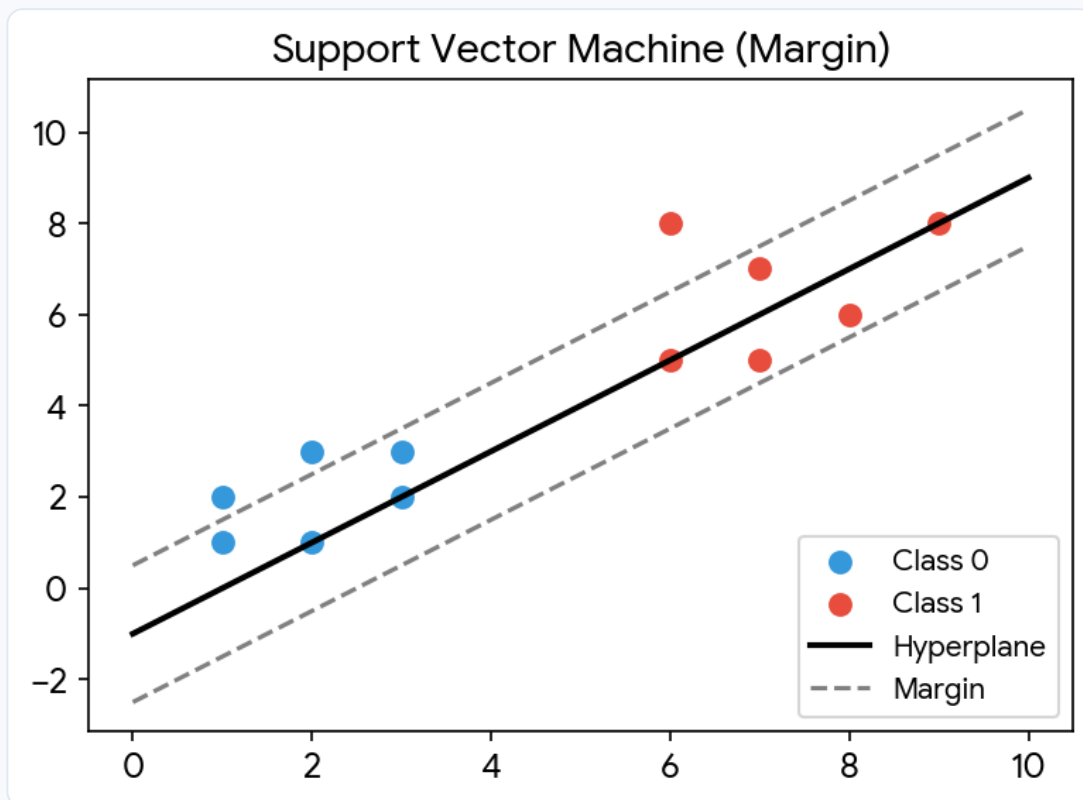


圖 3：支援向量機。黑線為最佳超平面，虛線為最大化 *Margin*，位於邊界上的點即為 *Support Vectors*。

核函數技巧 (Kernel Trick)： 這是 SVM 最精妙之處。當資料在低維度空間中無法被線性分割時（例如資料呈現同心圓分佈），SVM 透過核函數（如 RBF、Polynomial）將資料「映射」到高維度空間中。在那個高維空間裡，原本糾纏不清的資料突然就可以被一個簡單的平面切開了。運算上巧妙避開了高維空間的龐大計算量。

6. K-近鄰演算法 (K-Nearest Neighbors, KNN)

KNN 被稱為「惰性學習 (Lazy Learning)」演算法。它在「訓練階段」幾乎不做任何事情，只是單純地把所有訓練資料儲存起來。真正的計算發生在「預測階段」。

數學原理與核心機制

KNN 的哲學是「近朱者赤，近墨者黑」。當有一個未知的測試樣本輸入時，演算法會計算這個樣本與歷史資料庫中所有點的「距離」。最常用的距離測量標準是 **歐幾里得距離 (Euclidean Distance)**：

$$d(p, q) = \sqrt{(\sum (p_i - q_i)^2)}$$

計算完距離後，它會選出距離最近的 **K** 個鄰居。接著，統計這 K 個鄰居的標籤。如果是分類任務，就採用多數決（K 通常設定為奇數以避免平手）；如果是迴歸任務，則計算這 K 個鄰居數值的平均。

實務應用案例：在地化商品推薦與用戶相似度計算

某影音串流平台想為一位新註冊的用戶推薦電影。系統會將該用戶看過的少數幾部電影特徵轉化為空間中的一個座標點。接著，使用 KNN 演算法，在數百萬用戶庫中尋找距離這位新用戶「最近的 5 位老用戶 (K=5)」。由於這 5 位老用戶的觀影品味與新用戶在空間上高度重合，系統就會直接將這 5 位老用戶看過且按讚、但新用戶還沒看過的電影推薦給他。這是協同過濾 (Collaborative Filtering) 的基礎應用。

7. 單純貝氏分類器 (Naive Bayes)

單純貝氏是建立在機率論基礎上的高效分類器，特別在自然語言處理 (NLP) 領域有著不可撼動的地位。其核心立基於著名的「貝氏定理 (Bayes' Theorem)」。

數學原理與核心機制

貝氏定理描述了在已知某些條件發生的情況下，某事件的後驗機率：

$$P(A|B) = [P(B|A) * P(A)] / P(B)$$

在機器學習中，A 是類別標籤（如：垃圾郵件），B 則是觀察到的特徵集合（如：郵件內文包含「免費」、「中獎」）。這使得我們可以利用歷史資料（先驗機率 $P(A)$ 與概似度 $P(B|A)$ ），來反向推斷新特徵屬於哪個類別的機率。

之所以稱之為「**單純 (Naive)**」，是因為演算法做了一個極度強大卻在現實中極少成立的假設：**所有特徵之間都是條件獨立的**。這意味著它假設「免費」與「中獎」這兩個詞在信中出現的機率是互不影響的。儘管這個假設過於天真，但單純貝氏在巨量文本特徵下，計算速度如同閃電，且分類準確率出奇地高。

第三階段：非監督學習與複雜模式 (Unsupervised & Complex Patterns)

8. K-平均分群 (K-Means Clustering)

離開了有標準答案（標籤）的監督式學習，我們來到了非監督式學習的領域。K-Means 是最經典的分群演算法，其任務是將雜亂無章的資料，自動歸納出具備相似性的 K 個群體。

數學原理與核心機制

K-Means 是一個反覆迭代優化的演算法，步驟如下：

1. **初始化 (Initialization)**：隨機在資料空間中放置 K 個點，作為初始的「質心 (Centroids)」。
2. **分配 (Assignment)**：計算每一個資料點到這 K 個質心的距離，並將該資料點歸類給距離它最近的質心所屬的群體。
3. **更新 (Update)**：當所有點都分配完畢後，重新計算每一個群體內所有資料點的「幾何中心（平均座標）」，並將質心移動到這個新位置。
4. **收斂 (Convergence)**：不斷重複步驟 2 與 3，直到質心的位置不再發生變動，或達到設定的最高迭代次數為止。此時演算法達成了最小化群內變異 (WCSS) 的目標。

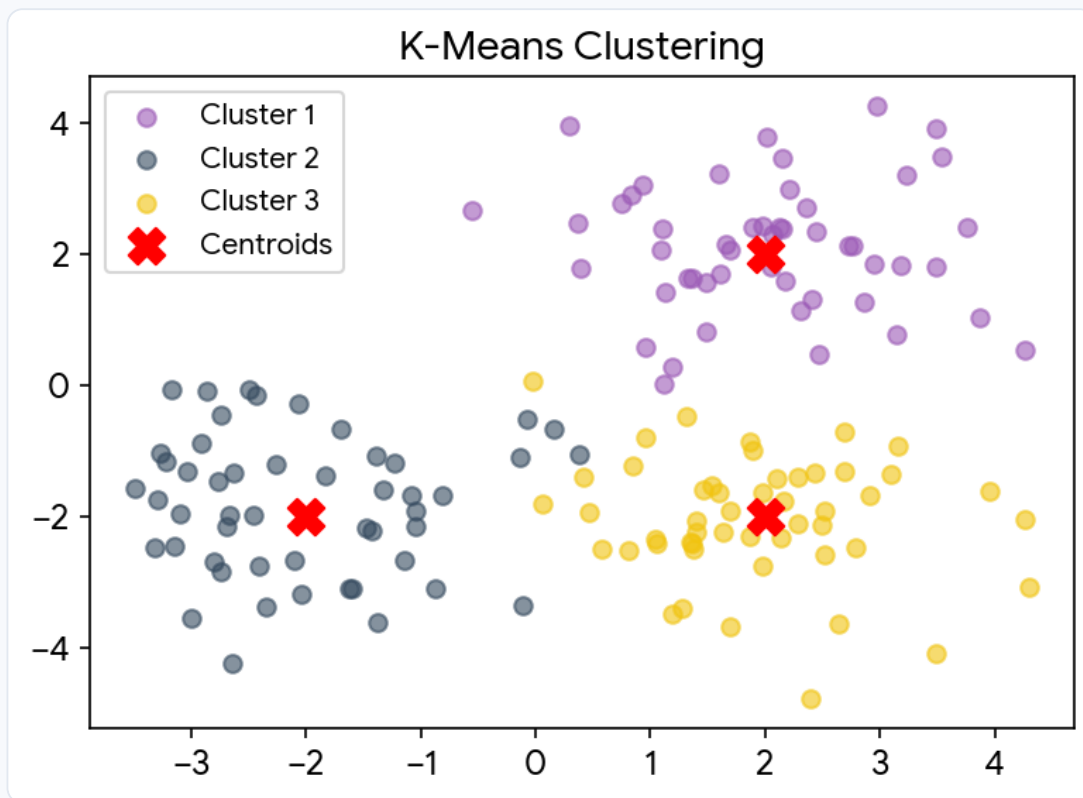


圖 4：K-Means 分群結果。算法將原本無標籤的點群，依據空間距離自動聚合為三個具備內在結構的子群體。

9. 主成分分析 (Principal Component Analysis, PCA)

在現實的大數據分析中，我們經常面臨「維度災難 (Curse of Dimensionality)」。當資料擁有成百上千個特徵時，不僅計算緩慢，還充滿雜訊。PCA 是一種強大的「降維 (Dimensionality Reduction)」技術。

數學原理與核心機制

PCA 的核心概念是透過正交轉換 (Orthogonal Transformation)，將原始的高維度、可能存在相關性的變數，轉換為一組低維度、線性獨立 (互不相關) 的變數，這些新變數被稱為「主成分 (Principal Components)」。

在數學上，這涉及計算資料共變異數矩陣 (Covariance Matrix) 的特徵值 (Eigenvalues) 與特徵向量 (Eigenvectors)。演算法會尋找資料中變異最大 (也就是包含最多資訊) 的方向作為第一主成分；接著在與第一主成分垂直的方向中，尋找變異次大的作為第二主成分，依此類推。我們只需保留前幾個累積變異貢獻率高的主成分，就能在捨棄大量雜訊維度的同時，保留原始資料 90% 以上的核心特徵。

第四階段：大腦啟發式建模 (High-Level Neural Networks)

10. 類神經網路與深度學習 (Neural Networks & Deep Learning)

作為目前人工智慧領域的霸主，類神經網路突破了傳統機器學習在特徵工程與複雜非線性問題上的瓶頸。它的架構靈感直接來源於人類大腦神經元的連結方式。

數學原理與核心機制

一個標準的前饋神經網路 (Feedforward Neural Network) 包含三個主要層次：

- **輸入層 (Input Layer)**：負責接收原始資料（如影像的像素、文本的向量）。
- **隱藏層 (Hidden Layers)**：介於輸入與輸出之間，負責特徵的抽象與學習。當網路的隱藏層數量非常多時，我們就稱之為「深度學習 (Deep Learning)」。
- **輸出層 (Output Layer)**：給出最終的預測結果或分類機率。

神經元的運作： 每個神經元會接收來自前一層神經元的訊號，將這些訊號與對應的「權重 (Weights)」相乘後加總，再加上「偏差 (Bias)」，最後通過一個**激勵函數 (Activation Function，如 ReLU, Sigmoid)** 引入非線性轉換，再傳遞給下一層。

$$z = \sum (w_i * x_i) + b$$
$$\text{Output} = \text{Activation}(z)$$

反向傳播 (Backpropagation) 演算法： 這是神經網路學習的靈魂。在訓練過程中，網路會先進行前向傳遞得出預測值，然後計算預測值與真實值之間的損失 (Loss)。接著，利用微積分中的「連鎖律 (Chain Rule)」，從輸出層反向將誤差一路傳遞回輸入層，計算出每個權重對這個誤差的貢獻（梯度），最後透過梯度下降法 (Gradient Descent) 稍微調整權重，讓下一次預測更準確。經過幾十萬次的訓練迭代，神經網路就能學會極度複雜的模式識別。

實務應用案例：自動駕駛視覺感知系統

特斯拉 (Tesla) 或 Waymo 的自動駕駛汽車，其核心視覺系統依賴深度的卷積神經網路 (CNN)。汽車上裝配的攝影機每秒傳回大量的影像畫面（輸入層）。神經網路的底層會先學習辨識簡單的邊緣與線條；中間層逐漸組合這些線條辨識出車輪、標誌牌形狀；到了高階的隱藏層，網路已經能完整理解畫面中的複雜概念：「前方有一位行人正在過馬路，且左側有一輛汽車突然切入」。神經網路能處理這種傳統演算法無法解決的高維度、非結構化圖像資料，做成即時的駕駛決策。

結語與學習建議 (Conclusion & Learning Roadmap)

機器學習的宇宙浩瀚無垠，本報告所探討的十大核心演算法，構建了資料科學家分析問題的強大武器庫。在實際應用中，並沒有所謂「完美」的演算法（即沒有免費午餐定理 No Free Lunch Theorem），選擇演算法取決於您的資料規模、特徵類型、預測準確度要求以及模型可解釋性的需求。

給初學者的三步進階指南：

1. **穩固基石 (STEP 1: Simple Relations)**：務必從「線性迴歸」與「邏輯迴歸」開始，徹底理解權重、偏差、損失函數與梯度下降等基礎觀念。
2. **掌握結構 (STEP 2: Classification & Clustering)**：學習決策樹、隨機森林以處理異質表格資料；學習 K-Means 與 PCA 以理解資料的潛在結構；熟悉 SVM 與 KNN 的空間幾何概念。
3. **挑戰深淵 (STEP 3: Optimization & Complexity)**：當傳統模型無法處理影像、語音或海量非結構化資料時，才踏入類神經網路與深度學習的領域，探索 CNN, RNN 與現代的 Transformer 架構。

— 本研讀報告結束，感謝您的深度閱讀與學習 —